TikuOS Test Suite

Simple. Ubiquitous. Intelligence, Everywhere.

Ambuj Varshney
ambuj@tiku-os.org

March 14, 2026

Outline

1. Why a Test Framework?
2. Architecture Overview
3. Test Harness
4. Output Format
5. Test Categories
6. TikuKits Integration
7. Configuration & Running Tests
8. Host-Native Testing
9. Python Test Runner
10. Test Specification Files
11. File Organization
12. Summary

Testing on Bare-Metal MCUs is Hard

Challenges:

- No OS, no test runner, no `stdout`
- Tests must run on real hardware over UART
- Limited flash (48–256 KB) and SRAM (2–8 KB)
- Subsystems interact with ISRs, watchdogs, MPU
- Optional libraries (tikukits) may or may not be present

TikuOS Test Framework Goals:

- Single shared harness header
- Machine-parseable output over UART
- Human-readable at the same time
- Per-module enable flags (no wasted flash)
- Compile-time mutual exclusion
- Optional library integration (tikukits)
- Runs on-target **and** host-native

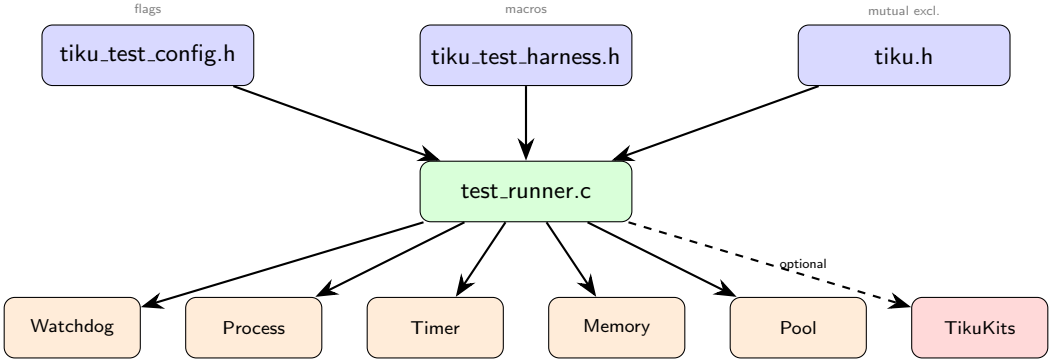
No Standard Framework Fits

Host-side test frameworks (CUnit, Unity) assume `malloc`, `printf`, and a hosted environment. TikuOS needs a zero-overhead, UART-friendly harness.

Key Insight

Embed the test protocol **in the output format** itself — no host-side agent needed, just parse UART lines.

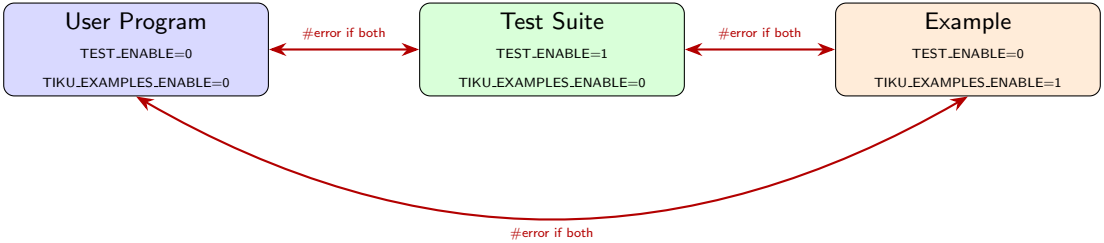
Test Framework Architecture



Three Header Layers

tiku_test_config.h — per-module enable flags **tiku_test_harness.h** — assertion & output macros
tiku.h — mutual exclusion guards

Firmware Modes: Mutual Exclusion



tiku.h — mode guard

```
#if TEST_ENABLE && TIKU_EXAMPLES_ENABLE
#error "TEST_ENABLE and "
  "TIKU_EXAMPLES_ENABLE "
  "cannot both be 1"
#endif
```

Three Firmware Modes

Exactly **one** mode at compile time:

1. **User program** — production firmware
2. **Test suite** — one test category active
3. **Example** — one example demo active

The compiler enforces this — no runtime check needed

Test Category Mutual Exclusion

tests/tiku_test_config.h — only one category at a time

```
#define _TEST_CATEGORY_COUNT                                                    \
    (!!TEST_WATCHDOG + !!TEST_CPUCLOCK + !!TEST_PROCESS + !!TEST_TIMER +      \
     !!TEST_MEM + !!TEST_PERSIST + !!TEST_MPU + !!TEST_POOL +                \
     !!TEST_REGION + !!TEST_KITS_MATRIX + !!TEST_KITS_STATISTICS +           \
     !!TEST_KITS_DISTANCE + !!TEST_KITS_SENSOR + !!TEST_KITS_SIGFEATURES + \
     !!TEST_KITS_TEXTCOMPRESSION)

#if TEST_ENABLE && (_TEST_CATEGORY_COUNT > 1)
#error "Only one test category may be enabled at a time"
#endif
```

Why Only One Category?

- MCU flash is limited (48–256 KB) — including all test code would overflow
- Linker `--gc-sections` eliminates dead code, but only if sections are truly unreferenced
- Some tests reinitialize subsystems (process, timer, watchdog) — running two would conflict
- Compile-time `#error` catches mistakes before flashing

Shared Test Harness: tiku_test_harness.h

Platform-aware print

```
#ifdef PLATFORM_MSP430
#include "tiku.h"
#define TEST_PRINT(...) \
    TIKU_PRINTF(__VA_ARGS__)
#else
#define TEST_PRINT(...) \
    printf(__VA_ARGS__)
#endif
```

Global counters

```
extern int tests_run;
extern int tests_passed;
extern int tests_failed;
```

Assertion macro

```
#define TEST_ASSERT(cond, msg)        \
  do {                                \
    tests_run++;                      \
    if (cond) {                       \
      tests_passed++;                 \
      TEST_PRINT(                     \
        "[T:P:%d] %s\n",             \
        tests_run, msg);              \
    } else {                          \
      tests_failed++;                 \
      TEST_PRINT(                     \
        "[T:F:%d] %s\n",             \
        tests_run, msg);              \
    }                                 \
  } while (0)
```

Single Source of Truth

All test modules include this one header — no duplicate macro definitions across modules.

No abort() or exit()

On an MCU, there is nowhere to exit *to*. Every assertion runs — no early termination.

Suite and Group Markers

Suite markers — bracket the entire test run

```
#define TEST_SUITE_BEGIN(name)                                          \
    do {                                                                \
        tests_run    = 0;                                               \
        tests_passed = 0;                                               \
        tests_failed = 0;                                               \
        TEST_PRINT("[TS:BEGIN] %s\n", name);                           \
    } while (0)

#define TEST_SUITE_END(name)                                            \
    TEST_PRINT("[TS:END] %s total=%d pass=%d fail=%d\n",               \
               name, tests_run, tests_passed, tests_failed)
```

Group markers — bracket a logical test function

```
#define TEST_GROUP_BEGIN(name)   TEST_PRINT("[TG:BEGIN] %s\n", name)
#define TEST_GROUP_END(name)    TEST_PRINT("[TG:END] %s\n", name)
```

Three-Level Hierarchy

Suite ⊃ **Group** ⊃ **Assertion** — suites reset counters, groups organize related assertions, and each assertion emits a pass/fail tag.

Machine-Parseable Output Protocol

Tag	Direction	Meaning
`[TS:BEGIN] name`	Start	Test suite started; counters reset to zero
`[TG:BEGIN] name`	Start	Test group (logical function) started
`[T:P:N] msg`	Result	Assertion `N` **passed**
`[T:F:N] msg`	Result	Assertion `N` **failed**
`[TG:END] name`	End	Test group ended
`[TS:END] name total=T pass=P fail=F`	End	Suite summary

Design Choices

- Square-bracket tags are easy to `grep` from mixed UART output
- `N` is a monotonically increasing counter — detects dropped lines
- The `[TS:END]` line is self-contained: a single regex extracts total/pass/fail
- Human-readable *and* machine-parseable — no separate verbose/quiet modes

Example UART Output

Captured from MSP430FR5969 over UART @ 115200 baud

```
[TS:BEGIN] TikuOS
[TG:BEGIN] Matrix Init
[T:P:1] init 2x3 returns OK
[T:P:2] rows == 2
[T:P:3] cols == 3
[T:P:4] all elements zero after init
[TG:END] Matrix Init
[TG:BEGIN] Matrix Identity
[T:P:5] identity 3x3 returns OK
[T:P:6] diagonal elements are 1.0
[T:P:7] off-diagonal elements are 0.0
[TG:END] Matrix Identity
  ...
[TG:BEGIN] Matrix Null Inputs
[T:P:110] null matrix rejected
[T:P:111] null data buffer rejected
[T:P:112] zero rows rejected
[TG:END] Matrix Null Inputs
[TS:END] TikuOS total=112 pass=112 fail=0
```

Parsing the Summary

A CI script only needs: `grep '^\[TS:END\]'` → extract `fail=N` → exit code

Parsing for Automated Testing

Shell: check pass/fail

```
# Capture UART output to file
make monitor > uart.log &
sleep 10 && kill %1

# Extract summary line
SUMMARY=$(grep '^\[TS:END\]' uart.log)

# Parse fail count
FAIL=$(echo "$SUMMARY" | \
  sed 's/.*fail=\([0-9]*\)/\1/')

if [ "$FAIL" -ne 0 ]; then
  echo "FAILED: $FAIL assertions"
  exit 1
fi
echo "ALL TESTS PASSED"
```

Python: parse all results

```
import re

with open("uart.log") as f:
    for line in f:
        m = re.match(
            r'\[T:([PF]):(\d+)\] (.+)',
            line)
        if m:
            status = m.group(1)
            num    = int(m.group(2))
            msg    = m.group(3)
            print(f"{'PASS' if status=='P'"
                  f" else 'FAIL'}"
                  f" #{num}: {msg}")
```

Extensible

The tagged format supports future tooling: dashboards, regression databases, or diff-based failure analysis.

Core OS Test Modules

Module	Config Flag	Tests	Covers
Watchdog	`TEST_WATCHDOG`	4	kick, pause/resume, interval, timeout
Process	`TEST_PROCESS`	10	lifecycle, events, yield, broadcast, poll, queue, local storage
Timer	`TEST_TIMER`	6	event, callback, periodic, stop, hardware timer
Arena	`TEST_MEM`	9	create, alloc, alignment, full, reset, peak, secure wipe
Pool	`TEST_POOL`	14	create, alloc/free, exhaustion, LIFO, peak, poisoning
Persist	`TEST_PERSIST`	12	init, register, read/write, delete, reboot, wear
MPU	`TEST_MPU`	11	init, lock/unlock, permissions, scoped write, violations
Region	`TEST_REGION`	12	init, contains, boundary, overflow, claim/unclaim
CPU Clock	`TEST_CPUCLOCK`	1	clock output pin for oscilloscope verification

79 Core OS Tests

Each module has its own header, source file(s), and fine-grained sub-flags. Auto-derived flags (`TEST_MEM`, `TEST_PROCESS`, etc.) aggregate sub-flags via `||` expressions.

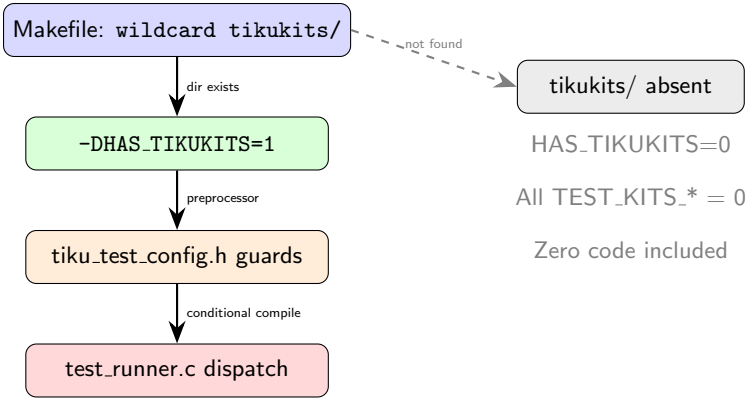
TikuKits Library Tests

Module	Config Flag	Functions	Covers
Matrix	`TEST_KITS_MATRIX`	12	init, identity, get/set, add/sub, mul, scale, det, trace
Statistics	`TEST_KITS_STATISTICS`	8	windowed, Welford, min/max, EWMA, energy, isqrt
Distance	`TEST_KITS_DISTANCE`	6	Manhattan, Euclidean, dot, cosine, Hamming
Sensor	`TEST_KITS_SENSOR`	2	fractional conversion, sensor name lookup
Sig. Features	`TEST_KITS_SIGFEATURES`	8	peak, ZCR, histogram, delta, Goertzel, z-score, scale
Compression	`TEST_KITS_TEXTCOMPRESSION`	4	RLE, byte-pair, heatshrink round-trip

TikuKits is Optional

All tikukits test flags are guarded by `#if defined(HAS_TIKUKITS)`. When the `tikukits/` directory is absent, all flags default to 0 and the compiler never sees tikukits headers or sources.

Optional Library Detection



Guard Chain

Makefile auto-detects → sets -D flag → config header enables/disables flags → test runner conditionally compiles. TikuOS always builds independently.

Makefile: Conditional Source Inclusion

Makefile — tikukits sources (library + tests)

```
HAS_TIKUKITS := $(if $(wildcard $(PROJ_DIR)/tikukits),1,0)
ifeq ($(HAS_TIKUKITS),1)
CFLAGS += -DHAS_TIKUKITS=1

# Library sources (only modules actually used get linked)
SRCS += $(wildcard tikukits/maths/linear_algebra/*.c)
SRCS += $(wildcard tikukits/maths/statistics/*.c)
SRCS += $(wildcard tikukits/maths/distance/*.c)
SRCS += $(wildcard tikukits/sensors/temperature/*.c)
SRCS += $(wildcard tikukits/sigfeatures/*/*.c)
SRCS += $(wildcard tikukits/textcompression/*/*.c)

# Test sources (only when testing is enabled)
ifeq ($(HAS_TESTS),1)
SRCS += $(wildcard tikukits/tests/maths/*.c)
SRCS += $(wildcard tikukits/tests/sensors/*.c)
SRCS += $(wildcard tikukits/tests/sigfeatures/*.c)
SRCS += $(wildcard tikukits/tests/textcompression/*.c)
endif
endif
```

Dead Code Elimination

Enabling a Test

tests/tiku_test_config.h

```
/* Step 1: Enable master switch */
#define TEST_ENABLE 1

/* Step 2: Enable ONE category */
#define TEST_KITS_MATRIX 1

/* All others stay 0 */
#define TEST_WATCHDOG 0
#define TEST_PROCESS_LIFECYCLE 0
#define TEST_MEM_CREATE 0
/* ... */
```

Build, flash, and monitor

```
# Build for MSP430FR5969
make clean
make MCU=msp430fr5969

# Flash via MSP-FET programmer
make flash MCU=msp430fr5969

# Monitor UART output
make monitor
```

Common Mistake

Forgetting `TEST_ENABLE = 1` — the category flag alone is not enough. The master switch gates `test_run_all()` in `main()`.

Auto-Derived Flags

You only set leaf flags (`TEST_MEM_CREATE`, `TEST_KITS_MATRIX`). Parent flags are computed automatically via `||` expressions:

- `TEST_MEM` = OR of all arena sub-flags
- `TEST_KITS_MATHS` = matrix | statistics | distance

Test Runner Dispatch

tests/test_runner.c — main dispatch

```c
void test_run_all(void)
{
    TEST_SUITE_BEGIN("TikuOS");

    /* Pre-interrupt tests */
    test_run_cpuclock();
    test_run_watchdog();
    test_run_process();
    test_run_mem();
    test_run_mpu();
    test_run_persist();
    test_run_pool();
    test_run_region();

    /* TikuKits library tests (no ISR dependency) */
#if defined(HAS_TIKUKITS)
    test_run_kits();
#endif

    /* Enable global interrupts (needed for timer ISRs) */
    __enable_interrupt();

    /* Post-interrupt tests */
    test_run_timer();
```

Writing a New Test

Example: a typical test function

```c
#include "tests/tiku_test_harness.h"

void test_my_feature(void)
{
    TEST_GROUP_BEGIN("My Feature");

    /* Setup */
    int result = my_function(42);

    /* Assertions -- each emits a tagged UART line */
    TEST_ASSERT(result == 84,
                "my_function doubles its input");
    TEST_ASSERT(my_function(0) == 0,
                "my_function handles zero");
    TEST_ASSERT(my_function(-1) == -2,
                "my_function handles negative");

    TEST_GROUP_END("My Feature");
}
```

Convention

One `TEST_GROUP_BEGIN/END` per function. Each `TEST_ASSERT` auto-increments the global counter and

Running Tests Without Hardware

Compile and run on host

```
gcc -std=c99 -Wall -Wextra -I. \
  tests/memory/test_mem_common.c \
  tests/memory/test_mem_arena.c \
  kernel/memory/tiku_mem.c \
  -o test_tiku_mem \
  && ./test_tiku_mem
```

Platform detection

```
#ifdef PLATFORM_MSP430
/* Real hardware paths */
#define TEST_PRINT(...) \
    TIKU_PRINTF(__VA_ARGS__)
#else
/* Host: plain printf */
#define TEST_PRINT(...) \
    printf(__VA_ARGS__)
#endif
```

Host Stubs

`test_mem_common.c` provides:

- `tiku_mem_arch_init()` — no-op
- `tiku_mem_arch_secure_wipe()` — volatile loop
- `tiku_mem_arch_nvm_read/write()` — memcpy
- Region table with test pools

Same Assertions, Same Output

The **exact same** `TEST_ASSERT` macro runs on both host and target. Output format is identical — the same parser works for both.

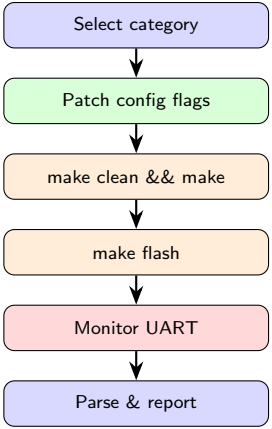
What Can Run on Host

Memory module tests (arena, pool, region

Automated Test Runner: tiku_test_runner.py

End-to-End Automation:

1. Interactive menu selects test category
2. Auto-detects TI LaunchPad serial port
3. Modifies `tiku_test_config.h` flags
4. Builds and flashes via `make`
5. Monitors UART, parses tagged output
6. Displays color-coded pass/fail results



Location

`tools/tiku_test_runner.py`
Supports 16 test categories across core OS and TikuKits.

Python Test Runner: Usage

Interactive mode

```
python3 tools/tiku_test_runner.py

# Select a test category:
#  1) Watchdog Timer
#  2) CPU Clock Output
#  3) Process/Protothread
#  4) Timer Subsystem
#  5) Arena Allocator
#  6) Pool Allocator
#  7) Persistent FRAM Store
#  8) MPU Write Protection
#  9) Region Registry
# ...
# Enter number: 3
```

Non-interactive / CI mode

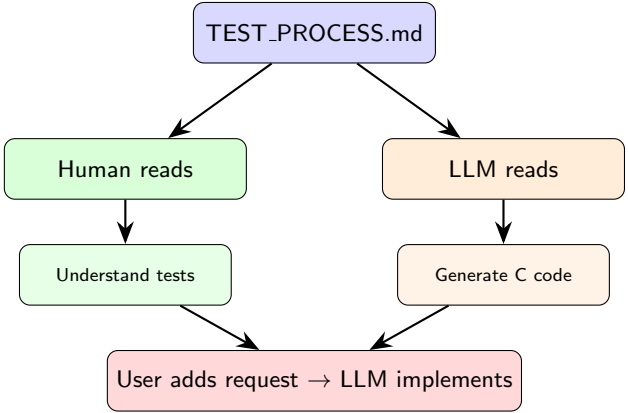
```
# Run specific category
python3 tools/tiku_test_runner.py \
  --category process \
  --mcu msp430fr5969

# Just patch config, don't build
python3 tools/tiku_test_runner.py \
  --category timer \
  --config-only

# Parse an existing log file
python3 tools/tiku_test_runner.py \
  --parse-file uart.log
```

Serial Auto-Detection

Scans for TI vendor ID (0451:2047) to find the LaunchPad's virtual COM port automatically.

Dual-Purpose Test Specifications

Each test module directory contains a `TEST_*.md` file that serves as both **documentation** and an **LLM instruction interface**:



Spec files created:

File	Tests
`TEST_WATCHDOG.md`	4
`TEST_CPUCLOCK.md`	1
`TEST_PROCESS.md`	10
`TEST_TIMER.md`	6
`TEST_MEMORY.md`	58
Total	**79**

Location Convention

`tests/<module>/TEST_<MODULE>.md`
Lives alongside the C source files it documents.

Spec File Structure

Each `TEST_*.md` file has three main sections:

1. **Overview**
   - Suite name, source directory, header file
   - Parent flag and hardware dependencies
   - Key constants used by the tests
2. **Tests** (one entry per test)
   - Config flag, source file, function name
   - Group name (as appears in UART output)
   - What the test validates and **why**
   - Key parameters and timing values
   - Every `TEST_ASSERT` listed with its message
3. **Requests** (user-editable)
   - Plain-English descriptions of new tests
   - LLM reads these to generate C code

Example: test entry in TEST_PROCESS.md

```
### 4. Broadcast Events

| Field | Value |
|-------|-------|
| **Flag** | ‘TEST_PROCESS_BROADCAST‘ |
| **Source** | ‘test_process_broadcast.c‘ |
| **Function** | ‘test_process_broadcast()‘ |
| **Group** | "Broadcast Events" |

**What it tests:**
Two processes (A and B) are started.
A broadcast event is posted. Both must
receive it. A second broadcast verifies
repeated delivery.

**Assertions:**
1. Process A received broadcast
2. Process B received broadcast
3. Both received second broadcast
```

The Request Workflow

1. User writes request in Requests section

Plain English, any detail level

2. LLM reads .md + .c files understands existing patterns

Spec gives full context: flags, assertions, patterns, naming

3. LLM generates new .c file updates config flag

Follows TikuOS conventions from spec + existing code

4. Request moves to Tests section with full docs

Spec stays accurate & current

Request Example: Adding a New Process Test

Step 1: User adds to `TEST_PROCESS.md`:

Requests section in TEST_PROCESS.md

```
### Request: Process re-start after exit

Start a process, let it run to
PROCESS_END (auto-exit), then start it
again. Verify:
- Second start delivers a fresh INIT
- Process runs from the beginning
- Phase counter resets properly
- is_running is correct throughout
```

Step 2: LLM reads the spec and sees:

- Existing pattern from lifecycle test
- Flag naming: `TEST_PROCESS_RESTART`
- File naming: `test_process_restart.c`
- Header: `test_process.h`
- Group naming convention

Step 3: LLM generates `test_process_restart.c`:

Generated: follows existing conventions

```c
#include "test_process.h"
#if TEST_PROCESS_RESTART

static volatile unsigned int
    restart_phase = 0;
TIKU_PROCESS(test_restart_proc,
             "test_restart");
TIKU_PROCESS_THREAD(test_restart_proc,
                    ev, data)
{
    TIKU_PROCESS_BEGIN();
    restart_phase++;
    TIKU_PROCESS_END();
}

void test_process_restart(void)
{
    TEST_GROUP_BEGIN("Process Restart");
    restart_phase = 0;
    tiku_process_init();
    /* First run */
    tiku_process_start(
        &test_restart_proc, NULL);
    while (tiku_process_run()) {}
    TEST_ASSERT(restart_phase == 1,
        "First run reached phase 1");
    /* Re-start */
```

Why This Approach Works

For Humans

- Complete test catalog in one place
- Every assertion documented with its purpose
- Quick reference: flag $\leftrightarrow$ file $\leftrightarrow$ function
- Destructive tests clearly marked

For LLMs

- Full context without reading all C files
- Naming conventions and patterns made explicit
- Existing test inventory prevents duplicates
- Request section is a clear instruction interface

Keeps Specs In Sync

The spec file lives **next to** the C files:

- Same PR updates both code and docs
- Spec is the review checklist for test PRs
- Stale entries are obvious (flag exists, no .c file)

Not a Replacement for Code

The `.md` files do not *generate* tests automatically — they are an **interface** that provides context and instructions. The C code remains the source of truth.

Test Framework Files

Layer	File	Role
Framework	`tests/tiku_test_config.h`	Per-module enable flags, mutual exclusion
	`tests/tiku_test_harness.h`	Assertion & output macros
	`tests/test_runner.c`	Central dispatch
	`tools/tiku_test_runner.py`	Automated build, flash, parse (16 categories)
Spec Files	`tests/watchdog/TEST_WATCHDOG.md`	Watchdog test spec (4 tests)
	`tests/cpuclock/TEST_CPUCLOCK.md`	CPU clock test spec (1 test)
	`tests/process/TEST_PROCESS.md`	Process test spec (10 tests)
	`tests/timer/TEST_TIMER.md`	Timer test spec (6 tests)
	`tests/memory/TEST_MEMORY.md`	Memory test spec (58 tests)
Core OS Tests	`tests/watchdog/*.c`	Watchdog timer tests (4 files)
	`tests/process/*.c`	Process/protothread tests (10 files)
	`tests/timer/*.c`	Timer subsystem tests (6 files)
	`tests/memory/*.c`	Arena, pool, persist, MPU, region (6 files)
TikuKits Tests	`tikukits/tests/maths/*.c`	Matrix, statistics, distance
	`tikukits/tests/sensors/*.c`	Sensor interface
	`tikukits/tests/sigfeatures/*.c`	Signal feature extraction
	`tikukits/tests/textcompression/*.c`	RLE, BPE, heatshrink
Guards	`tiku.h`	Tests vs examples mutual exclusion
	`examples/tiku_example_config.h`	Single-example guard

Summary

Test Framework Design

- Shared harness: `tiku_test_harness.h`
- Machine-parseable UART output
- Three-level hierarchy: suite ⊃ group ⊃ assertion
- Per-module compile-time enable flags
- Auto-derived parent flags via `||`
- Compile-time mutual exclusion (`#error`)
- Runs on-target and host-native

Automation & Tooling

- **Python test runner** — end-to-end: select, build, flash, parse
- **TEST *.md spec files** — dual-purpose

Test Coverage

- **79** core OS tests across 9 modules, each documented in spec files
- **40** tikukits test functions across 6 modules
- 5 spec files total: watchdog, cpuclock, process, timer, memory

Spec-Driven Development

Every test is documented with: flag, source file, what it validates, and every assertion. Users add plain-English requests; LLMs use the spec as context to generate correct, convention-following C test code.

TikuKits: Fully Optional

Thank You

TikuOS Test Suite

Testing Framework for Physical AI Endpoints

Repository:	`https://github.com/tikuos/tikuOS`
License:	Apache 2.0
Harness:	`tests/tiku_test_harness.h`
Config:	`tests/tiku_test_config.h`
Runner:	`tests/test_runner.c`
Python Runner:	`tools/tiku_test_runner.py`
Spec Files:	`tests/<module>/TEST_<MODULE>.md`